

[← Go Back](#)

Hardware

Nicla Voice

Tutorials

Door Intruder Detector
Using ML with the Nicla
Voice

Audio Analysis with
Machine Learning and the
Nicla Voice

Glass-Breaking Detection
Using Edge Impulse & Nicla
Voice

Motion Detection with Nicla
Voice and Machine
Learning Tools

Nicla Voice User Manual

Home / Hardware / Nicla Voice / **Nicla
Voice User Manual**

Nicla Voice User Manual

Learn about the hardware and software
features of the Arduino® Nicla Voice.

José Bagur,
Benjamin
Author · Dannegård and
Christopher
Méndez

Last
revision · 06/25/2025

ON THIS PAGE

Overview

Hardware and Software Requirements	+
Product Overview	+
First Use	+
Pins	+
Onboard Sensors	+
Actuators	+
Communication	+
Support	+

Overview

This user manual will provide you with a comprehensive overview of the Arduino Nicla Voice board, covering its main hardware and software features. This user manual will also show you how to set up, configure, and use these features.

Hardware and Software Requirements

Hardware Requirements

Nicla Voice (x1)

Micro USB cable (x1)

Software Requirements

Arduino IDE 1.8.10+, **Arduino IDE 2.0+**, or **Arduino Cloud Editor**

To create custom Machine Learning models, the integrated **Machine Learning Tools** of the **Arduino Cloud** are needed. In case you do not have an Arduino Cloud account, you will need to create one first.

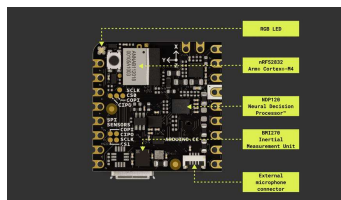
Product Overview

The Nicla Voice is an innovative and versatile development board designed by the Arduino team for sound-enabled projects and applications. This board has an onboard always-on speech recognition and sensor-fusion processor, advanced motion sensors, and wireless connectivity via Bluetooth® Low Energy. The Nicla Voice is an ideal solution for

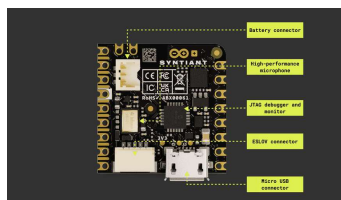
gesture or voice recognition systems to contactless wireless applications.

Board Architecture Overview

The Nicla Voice features a robust and efficient architecture integrating various components to enable speech, sound, and motion projects and applications.



The Nicla Voice main components (top view)



The Nicla Voice main components (bottom view)

Here is an overview of the board's architecture's main components shown in the images above:

Microcontroller: at the heart of the Nicla Voice is the nRF52832, a powerful and versatile System-on-Chip (SoC) from Nordic® Semiconductor. The nRF52832 is built around a 32-bit Arm® Cortex®-M4 processor running at 64 MHz.

Speech and sensor-fusion processor: the board features the NDP120 Neural Decision Processor™, an ultra-low power always-on audio and sensor-fusion processor from Syntiant®, which enables several applications, including echo-cancellation, beamforming, noise suppression, speech enhancement, speaker

classification, and multi-sensor fusion.

Onboard advanced motion

sensors: the board features the BMI270, a high-precision Inertial Measurement Unit (IMU) from Bosch® Sensortec, which combines a 3-axis accelerometer and a 3-axis gyroscope for precise motion tracking and orientation detection. The board also features the BMM150, a compact geomagnetic sensor from Bosch® Sensortec with a 3-axis magnetometer.

Onboard high-performance

microphone: the Nicla Voice is equipped with the IM69D130, a high-quality MEMS microphone by Infineon® Technologies. The IM69D130 offers excellent audio quality and low noise performance, ensuring accurate and distortion-free audio capturing.

Onboard connector for external

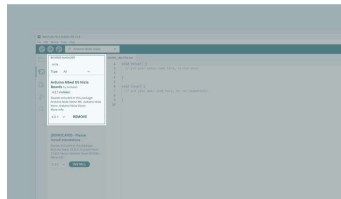
PDM microphone: an external PDM microphone can be connected to the board via an onboard connector.

Wireless connectivity: the board supports Bluetooth® Low Energy connectivity, provided by the ANNA-B112 module developed by u-blox®. This compact, high-performance Bluetooth® Low Energy module allows the Nicla Voice to communicate wirelessly with other devices and systems.

Power management: the Nicla Voice is designed for ultra-low power operation, with efficient power management features that ensure minimal energy consumption even when using always-on speech recognition and multiple sensors. The Nicla Voice features the BQ25120 from Texas Instruments®; a highly integrated battery charge management integrated circuit (IC) designed for wearables and Internet of Things (IoT) devices.

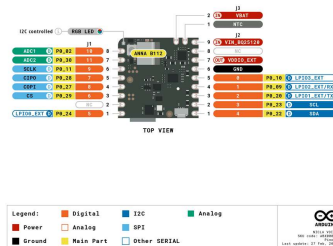
Board Core and Libraries

you need to work with the board's components, such as its IMU, magnetometer, and onboard microphone. To install the core for Nicla boards, navigate to **Tools > Board > Boards Manager** or click the Boards Manager icon in the left tab of the IDE. In the Boards Manager tab, search for `nicla` and install the latest **Arduino Mbed OS Nicla Boards** version.



Installing the Arduino Mbed OS Nicla Boards core in the Arduino IDE bootloader

Pinout



Nicla Voice pinout

The full pinout is available and downloadable as PDF from the link below:

[Nicla Voice pinout](#)

Datasheet

The complete datasheet is available and downloadable as PDF from the link below:

Schematics

The complete schematics are available and downloadable as PDF from the link below:

[Nicla Voice schematics](#)

STEP Files

The complete STEP files are available and downloadable from the link below:

[Nicla Voice STEP files](#)

First Use

Powering the Board

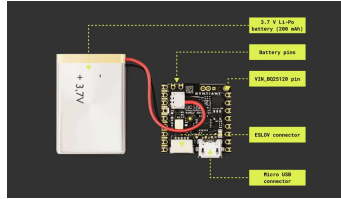
The Nicla voice can be powered by:

Using a Micro USB cable (not included).

Using an external **5V power supply** connected to `VIN_BQ25120` pin (please refer to the [board pinout section](#) of the user manual).

Using a **3.7V Lithium Polymer (Li-Po) battery** connected to the board through the onboard battery connector; the manufacturer part number of the battery connector is BM03B-ACHSS and its matching receptacle manufacturer part number is ACHR-03V-S. The **recommended minimum battery capacity for the Nicla Voice is 200 mAh**. A Li-Po battery with an integrated NTC thermistor is also recommended for thermal protection.

Using the onboard **ESLOV connector**, which has a dedicated 5V power line.



Different ways to power the
Nicla Voice

A 3.7V Li-Po battery can be also connected through the board's pins:



1 (NTC), 2 (VBAT), and
6 (GND).

Please refer to the board's [pinout](#) to locate those pins on your Nicla Voice board.

Onboard Battery Charger

The onboard battery charger of your board is, by default, **disabled**. To enable it, you can use the `enableCharging()` function defined in the Nicla Voice board core:

```
1 // Enable the onboard battery
2 // The function parameter def
3 nicla::enableCharging(100);
```

The desired charging current can be set to a value between 5 mA and 300 mA; the default value is 20 mA.

A safe default charging current value that works for most common LiPo batteries is 0.5C, which means

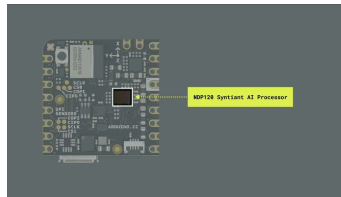


charging at a rate equal to half the battery's capacity. For example, a 200 mAh battery could be safely charged at 100 mA (0.1 A).

To disable the onboard battery charger, you can use the `disableCharging()` function defined in the Nicla Voice board

```
1 // Disable the onboard battery
2 nicla::disableCharging();
```

NDP120 Processor Firmware Update



NDP Syntiant AI Processor

It is recommended to update the NDP120 processor firmware and the built-in speech recognition model to the latest release. Follow these three steps to complete the update process:

- 1 Select the Nicla Voice board in the Arduino IDE so you can have access to the board built-in examples.
- 2 Upload the `Syntiant_upload_fw_ymodem` sketch. This sketch can be found in the board's built-in examples by navigating to **File > Examples > NDP > Syntiant_upload_fw_ymodem**.
- 3 After uploading the sketch, **format your board's external Flash memory** before uploading the updated NDP120 processor firmware files. You can do this by navigating to the Arduino IDE Serial Monitor and typing `F` and then `Enter`.

uploaders for various operating systems, and the updated NDP120 processor firmware and speech recognition model, in a known location on your computer.

- 5 Open a new terminal in the location where the .zip file was extracted and execute the following command:

```
1 syntiant-uploader-<OS
```

Replace `portName` and `filename` with the relevant information. Three different files must be uploaded to the board by executing the following three commands, for example on **Windows** the commands are the following:

```
1 ./syntiant-uploader-v
```

```
1 ./syntiant-uploader-v
```

```
1 ./syntiant-uploader-v
```

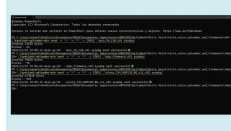
As we are using Windows, the command used is

 syntiant-uploader-win

Ensure all executed commands return a

```
filename sent successful
```

message in the console, as shown in the image below.



Uploader feedback messages

After uploading the three files, your board's firmware is updated to the latest release and ready to be used.

External Memory Format

Your board NDP120 processor files (firmware and models) are stored in your board's external Flash memory. It is recommended to **format your Nicla Voice external Flash memory** every time you are going to update the processor firmware or when you are going to update/add models to the external Flash memory.

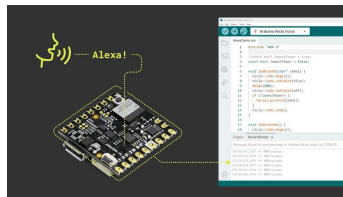
Follow these steps to perform the external memory format process:

- 1 Select the Nicla Voice board in the Arduino IDE so you can have access to the board built-in examples.
- 2 Upload the `Syntiant_upload_fw_ymodem` sketch. This sketch can be found in the board's built-in examples by navigating to **File > Examples > NDP > Syntiant_upload_fw_ymodem**.
- 3 After uploading the sketch, navigate to the IDE's Serial Monitor, type `F`, and press `Enter`. Your board's external memory should be formatted now, you can confirm this by

After completing this process, you can upload NDP processor's firmware and model files to your board's external memory without issues as explained before.

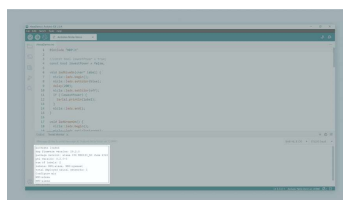
Built-in Speech Recognition Example

The speech recognition example can be found in the board's built-in examples by navigating to **File > Examples > NDP > AlexaDemo**.



Alexa speech recognition example

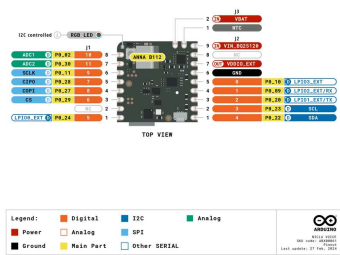
After successfully updating the NDP120 processor firmware and the speech recognition model to the latest release, we can upload the speech recognition example to the Nicla Voice. To test the example, say "Alexa"; this should make the onboard LED of the Nicla Voice blink blue if the keyword "Alexa" is recognized. If there is no response from the board, try speaking from a closer proximity or louder. You should also see in the Serial Monitor if the word "Alexa" was detected, as shown in the image below:



AlexaDemo example feedback in the Arduino IDE Serial Monitor

Pins

The Nicla Voice pinout is shown in the image below:



Nicla Voice pinout

Analog Pins

The Nicla Voice has **two analog input pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping
ADC1 / P0_02	A0
ADC2 / P0_30	A1

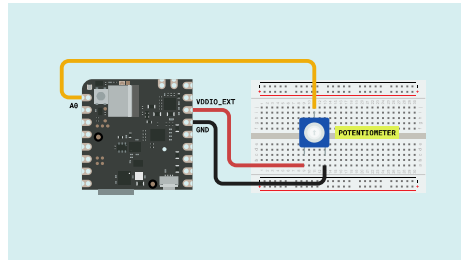
Both pins can be used through the built-in functions of the Arduino programming language.

Nicla boards ADC can be configured to 8, 10 or 12 bits defining the argument of the following function respectively (default is 10 bits):

```
1 analogReadResolution(12); //
```

i The Nicla boards ADC reference voltage is fixed to 1.8V, this means that it will map the ADC range from 0 to 1.8 volts.

connected to `A0` and displays it on the IDE Serial Monitor. To understand how to properly connect a potentiometer to the Nicla Voice pins, take the following image as a reference:



```
1 #include "Nicla_System.h"
2
3 int sensorPin = A0; // select the input pin
4 int sensorValue = 0; // variable to store the value coming from the sensor
5
6 void setup() {
7
8     analogReadResolution(12);
9     nicla::begin();
10    Serial.begin(115200);
11 }
12
13 void loop() {
14     // read the value from the sensor:
15     sensorValue = analogRead(sensorPin);
16     // print the value
17     Serial.println(sensorValue);
18     delay(1000);
19 }
```

The ADC inputs support 3.3V even when the ADC reference is 1.8V. In this perspective, the ADC will not sense any change from 1.8V and above.

Digital Pins

The Nicla Voice has **twelve digital pins**, mapped as follows:

Microcontroller Pin	Arduino Pin Mapping
---------------------	---------------------

Microcontroller Pin	Arduino Pin Mapping
P0_09	1
P0_20	2
P0_23	3
P0_22	4
P0_24	5
P0_29	6
P0_27	7
P0_28	8
P0_11	9
P0_02	A0
P0_30	A1

Notice that analog pins `A0` and `A1` (`P0_02` and `P0_30`) can also be used as digital pins. Please, refer to the [board pinout section](#) of the user manual to find them on the board.

The digital pins of the Nicla Voice can be used as inputs or outputs through the built-in functions of the Arduino programming language.

The Nicla Voice digital I/O's are low power, so to drive output devices like LEDs, resistive loads, buzzers, etc, it is recommended to use a MOSFET driver or a buffer to guarantee the required current flow. Learn more about the Nicla I/O's considerations [here](#).

The configuration of a digital pin is done in the `setup()` function with the built-in function `pinMode()` as shown below:

```

// Example code for pin configuration
void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

```

```
1 // Pin configured as an input
2 pinMode(pin, INPUT);
3
4 // Pin configured as an output
5 pinMode(pin, OUTPUT);
6
7 // Pin configured as an input
8 pinMode(pin, INPUT_PULLUP);
```

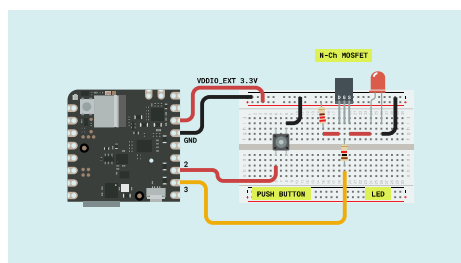
The state of a digital pin, configured as an input, can be read using the built-in function `digitalRead()` as shown below:

```
1 // Reads pin state, stores value
2 state = digitalRead(pin);
```

The state of a digital pin, configured as an output, can be changed using the built-in function `digitalWrite()` as shown below:

```
1 // Set pin on
2 digitalWrite(pin, HIGH);
3
4 // Set pin off
5 digitalWrite(pin, LOW);
```

The example code shown below uses digital pin `3` to control an LED and reads the state of a button connected to digital pin `2`:



```
1  #include "Nicla_System.h"
2
3  // Define button and LED p
4  int buttonPin = 2;
5  int ledPin = 3;
6
7  // Variable to store the b
8  int buttonState = 0;
9
10 void setup() {
11
12     nicla::begin();
13
14     // Configure button and
15     pinMode(buttonPin, INPUT);
16     pinMode(ledPin, OUTPUT);
17
18     // Initialize Serial com
19     Serial1.begin(9600);
20 }
21
22 void loop() {
23     // Read the state of the
24     buttonState = digitalRead(buttonPin);
25
26     // If the button is pres
27     if (buttonState == LOW)
28         digitalWrite(ledPin, HIGH);
29 }
```

PWM Pins

Most digital and analog pins of the Nicla Voice can be used as PWM (Pulse Width Modulation) pins. This functionality of the Nicla Voice pins can be used with the built-in function `analogWrite()` as shown below:

```
1  analogWrite(pin, value);
```

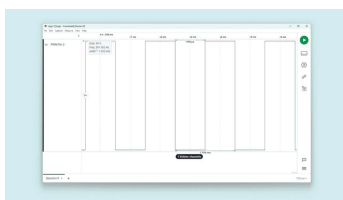
By default, the output resolution is 8 bits, so the output value should be between 0 and 255. To set a greater resolution, do it using the built-in function `analogWriteResolution()` as shown below:


```
1 analogWriteResolution(bits);
```

Here is a complete example code that outputs a 50% duty-cycle PWM signal through pin 3 of the Nicla Voice:

```
1 #include "Nicla_System.h"
2
3 #define out_pwm 3 // Nicla
4
5 void setup() {
6     // put your setup code here
7     nicla::begin(); // Nicla
8     analogWriteResolution(12)
9     analogWrite(out_pwm, 2048)
10 }
11
12 void loop() {
13     // put your main code here
14 }
```

Using this function has some limitations, for example, the PWM signal frequency is fixed at 500 Hz, and this could not be ideal for every application.



PWM output signal using
analogWrite()

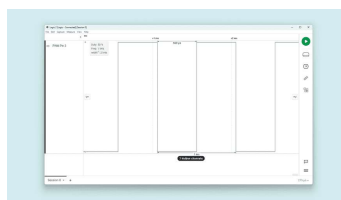
If you need to work with a higher frequency PWM signal, you must do it by working with the PWM peripheral at a lower level as shown in the example code below:

```

1  #include "Nicla_System.h"
2  #include "nrfx_pwm.h"
3
4  static nrfx_pwm_t pwm1 = N
5
6  static uint16_t /*const*/
7
8  static nrf_pwm_sequence_t
9    .values = { .p_common =
10    .length = NRF_PWM_VALUES,
11    .repeats = 0,
12    .end_delay = 0
13  };
14
15  void setup() {
16
17    nicla::begin();
18    nrfx_pwm_config_t config
19      .output_pins = {
20        32 + 23, // Nicla V
21      },
22      .irq_priority = APP_IR
23      .base_clock = NRF_PWM_
24      .count_mode = NRF_PWM_
25      .top_value = 1000,
26      .load_mode = NRF_PWM_L
27      .step_mode = NRF_PWM_S
28  };

```

The code above results in a 1KHz square waveform with a 50% duty cycle as in the image below. The frequency is defined by the `.base_clock` and `.top_value` variables, and the duty cycle by the `seq1_values` variable.



PWM output signal using the
PWM at a lower level

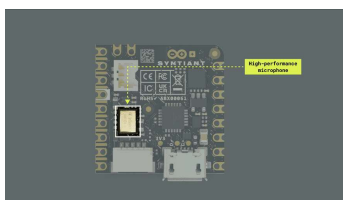
Onboard Sensors

The Nicla Voice boards come with various onboard sensors that allow you

microphone, a 6-axis IMU, and a 3-axis magnetometer. The onboard sensors can be used for developing various applications, such as voice recognition, gesture control, and environmental monitoring.

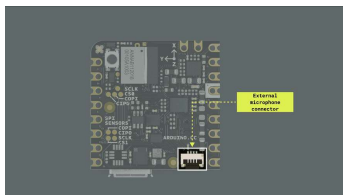
Microphone

The onboard high-performance microphone of the Nicla Voice is the IM69D130 from Infineon® Technologies. The IM69D130 is specifically designed for applications that require high-quality audio recording and accurate voice detection, such as voice-controlled Internet of Things (IoT) devices, smart home systems, and mobile devices.



Nicla Voice onboard high-performance microphone

An external PDM microphone can be connected to the board via an onboard Zero Insertion Force (ZIF) connector. The manufacturer part number of the onboard ZIF connector is FH33J-4S-0.5SH(99); a 4-pin 0.5 mm pitch FFC cable is compatible with this connector.



Nicla Voice onboard ZIF connector

The example code shown below captures audio from the onboard microphone of

compressed audio data to the serial port. The example can be found in the board's built-in examples by navigating to **File > Examples > NDP > Record_and_stream**.

Keep in mind that this example code requires the following libraries:

[arduino-libg722](#)

[arduino-audio-tools](#)

If you encounter compilation or build errors with the `Record_and_stream` example, please try using the `arduino-audio-tools` [library version 0.9.6](#).

Make sure to install these libraries in the Arduino IDE before uploading the code to your Nicla Voice board.

```
1 // Include necessary libra
2 #include "Arduino.h"
3 #include "NDP.h"
4 #include "AudioTools.h"
5 #include "AudioCodecs/Code
6
7 // Create an instance of t
8 G722Encoder encoder;
9
10 // Declare a buffer to tem
11 uint8_t data[2048];
12
13 // Define a function to tu
14 void ledGreenOn() {
15     nicla::leds.begin();
16     nicla::leds.setColor(gre
17     delay(200);
18     nicla::leds.setColor(off
19     nicla::leds.end();
20 }
21
22 void setup() {
23     // Start UART communicat
24     Serial.begin(115200);
25
26     // Initialize the Nicla
27     nicla::begin();
28     nicla::disableLDO();
```

Here you can find a step-by-step explanation of the code:

First, the necessary libraries are included:

`Arduino.h` and `NDP.h` for the Nicla Voice board's basic functions and microphone control; `AudioTools.h` and `CodecG722.h` for audio processing and G722 codec support.

An instance of the `G722Encoder` class is created to handle audio encoding.

The `data` buffer is declared with a size of 2048 bytes to store audio data temporarily.

In the `setup()` function:

The serial communication is initialized at a baud rate of 115200.

The green LED is configured to turn on when an event occurs.

The `G722Encoder` instance is set up with the proper audio parameters (1 channel, 16 kHz sample rate).

The output stream for the encoder is set to the serial port.

The NDP Processor is set up with the necessary firmware packages and the onboard microphone is turned on.

The audio chunk size is checked to ensure it doesn't exceed the buffer size.

In the `loop()` function:

Audio data is extracted from the NDP processor and stored in the `data` buffer.

The length of the extracted audio data is stored in the `len` variable.

The extracted audio data is passed to the G722 encoder, which compresses the audio and sends it to the serial port.

To extract the audio data on a **Linux computer**, you will need to set up the serial port as raw:

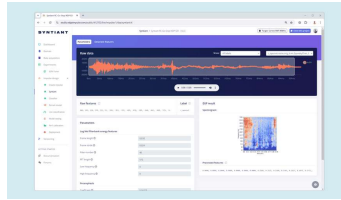
```
1 stty -F /dev/ttyACM0 115200
```

Dump the data to a file (e.g., test.g722):

```
1 cat /dev/ttyACM0 > test.g722
```

Then, you can open the file with a software like [Audacity](#) to play back the audio.

You can use the Nicla Voice and the **Machine Learning Tools** of the Arduino Cloud to create your own audio analysis Machine Learning models. Check out this **tutorial** and start with Machine Learning with the Nicla Voice.

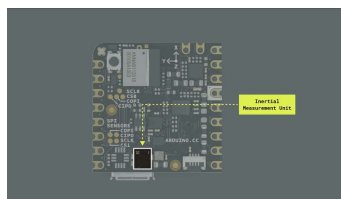


Edge Impulse Studio for ML
and AI

Learn more about the Nicla Voice integration with Edge Impulse [here](#).

IMU

The Nicla Voice features an advanced IMU and a magnetometer, which allows the board to sense motion, orientation, and magnetic fields. The IMU on the Nicla Voice board is the BMI270 from Bosch®. It consists of a 3-axis accelerometer and a 3-axis gyroscope. They can provide information about the board's motion, orientation, and rotation in a 3D space. The BMI270 is designed for wearables and offers low power consumption and high performance, making it suitable for various applications, such as gesture recognition, motion tracking, or stabilization.



Nicla Voice onboard IMU

Accelerometer and Gyroscope Data

The example sketch below shows how to get acceleration (m/s^2) and angular velocity (in $^\circ/\text{s}$) data from the onboard IMU and streams it to the Arduino IDE Serial Monitor and Serial Plotter. The sketch needs the `BMI270_Init.h` header file to be in the same directory as the sketch. You can download the example sketch and the header files [here](#).



For this example to work you must update the NDP processor, see the details on this [section](#).

```

1  #include "NDP.h"
2  #include "BMI270_Init.h"
3
4  // Named constants
5  #define READ_START_ADDRESS 0x00
6  #define READ_BYTE_COUNT 1
7  #define SENSOR_DATA_LENGTH 14
8
9  // Accelerometer range is +/- 2g
10 // Raw accelerometer data
11 // Raw accelerometer data
12 #define ACCEL_SCALE_FACTOR 16384
13
14 // Gyroscope has a sensitivity of 131 degrees per second
15 #define GYRO_SCALE_FACTOR 131
16
17 // Macros for checking the status of the IMU
18 #define CHECK_STATUS(s) \
19     do { \
20         if (s) { \
21             Serial.print("SPI a"); \
22             Serial.println(__LINE__); \
23             for (;;) \
24                 ; \
25         } \
26     } while (0)
27
28 void setup() {

```

Here you can find a step-by-step explanation of the code:

First, the necessary libraries are included:

`NDP.h` and `BMI270_Init.h` for the Nicla Voice board's basic functions and the IMU control.

Macros are defined for checking the status of the IMU; these macros allow the sketch to detect and handle sensor errors.

Next, in the `setup()` function:

The serial communication is initialized at a baud rate of `115200`.

The Nicla Voice board is initialized, and the LDO regulator (used for putting the board into power-saving mode) is disabled to avoid communication problems with the IMU.

Error and event handlers are initialized.

NDP processor is initialized; this process includes populating the external Flash memory of the board with the NDP processor's internal microcontroller firmware (`mcu_fw_120_v91.synpkg`), and the NDP processor's internal DSP firmware (`dsp_firmware_v91.synpkg`).

The BMI270 sensor is initialized; this includes a software reset, loading the sensor configuration, and setting it into normal power mode with the accelerometer and gyroscope operational.

Finally, in the `loop()` function:

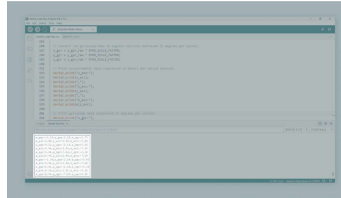
Memory is allocated for the sensor data; data is then read from the sensor and stored in this allocated space.

Raw sensor data is then parsed and extracted into raw accelerometer and gyroscope data. This data is represented as 16-bit signed integers ranging from -32768 to 32767.

Raw sensor data is converted into understandable and standard unit measurements; for the accelerometer, data is converted to meters per second squared, and for the gyroscope, data is

Converted accelerometer and gyroscope data are printed on the Serial Monitor, allowing the user to observe sensor data in real-time.

After uploading the example code, you should see accelerometer and gyroscope data on the IDE's Serial Monitor as shown below:



Nicla Voice onboard IMU data on the IDE's Serial Monitor

Let's use also the Arduino IDE Serial Plotter to test the example IMU sketch, open the IDE's Serial Plotter by navigating to **Tools > Serial Plotter**. After a while, you should see a real-time graph showing data from the board's onboard accelerometer and gyroscope, as shown below:



When the board is not moving, you should see acceleration measurements close to zero on the x and y-axis, while the z-axis will be close to 1g (approximately 9.81 m/s^2), the gyroscope measurements on the three-axis will stay close to zero.

IMU and Machine Learning

The example code below demonstrates how to use the Nicla Voice board to

indicators using the onboard RGB LED and sends IMU data to the NDP processor for inference. The example can be found in the board's built-in examples by navigating to **File > Examples > NDP > IMUDemo**.

```
1  #include "NDP.h"
2
3  // Set to 'true' for the l
4  const bool lowestPower = f
5
6  // Function to turn on the
7  void ledBlueOn(char* label
8      nicla::leds.begin();
9      nicla::leds.setColor(blue
10     delay(200);
11     nicla::leds.setColor(off
12     if (!lowestPower) {
13         Serial.println(label);
14     }
15     nicla::leds.end();
16 }
17
18 // Function to turn on the
19 void ledGreenOn() {
20     nicla::leds.begin();
21     nicla::leds.setColor(green
22     delay(200);
23     nicla::leds.setColor(off
24     nicla::leds.end();
25 }
26
27 // Function to make the re
28 void ledRedBlink() {
```

In the example code above, a Machine Learning model is loaded into the Nicla Voice board, and predefined IMU data is sent to the Machine Learning model for inferencing. Depending on the result, the board will light its built-in RGB LED with different colors:

If the model matches the input data with a known motion pattern, the built-in RGB LED is turned blue, and the event label is printed to the IDE's Serial Monitor.

If an error occurs, the built-in RGB

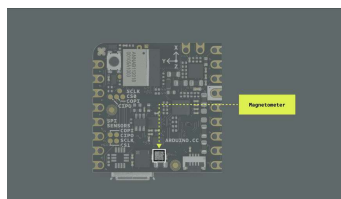
While an event is recognized, the built-in RGB LED is turned on green.

To learn more about your Nicla Voice board Machine Learning capabilities, check out the following tutorial and learn how to create a simple motion detection application:

Motion Detection with Nicla Voice and Machine Learning Tools

Magnetometer

The onboard magnetometer of the Nicla Voice can be used to determine the board's orientation relative to Earth's magnetic field, which is helpful for compass applications, navigation, or detecting the presence of nearby magnetic objects. The magnetometer on the Nicla Voice board is the BMM150, also from Bosch®. It is a 3-axis sensor that measures the strength and direction of magnetic fields surrounding the board.



Nicla Voice onboard magnetometer

Magnetometer Data

The example code below shows how to get raw magnetic field and Hall resistance data from the onboard magnetometer and stream it to the Arduino IDE Serial Monitor and Serial Plotter. You can download the example sketch [here](#).

```

1  #include "NDP.h"
2
3  // Named constants
4  #define READ_START_ADDRES
5  #define READ_BYTE_COUNT 8
6  #define SENSOR_DATA_LEN
7
8  /**
9   * Blinks onboard red LED
10  */
11  void ledRedBlink() {
12    while (1) {
13      nicla::leds.begin();
14      nicla::leds.setColor(
15        delay(200);
16      nicla::leds.setColor(
17        delay(200);
18      nicla::leds.end();
19    }
20  }
21
22  // Macro definition used
23  #define CHECK_STATUS(s) \
24    do { \
25      if (s) { \
26        Serial.print("SPI a
27        Serial.println(__LI
28        for (;;) \

```

Here you can find a step-by-step explanation of the sketch:

First, the necessary libraries are included:

`NDP.h` for the Nicla Voice board's basic functions and the magnetometer control.

Macros are defined for checking the status of the magnetometer; these macros allow the sketch to detect and handle sensor errors.

Next, user function `ledRedBlink()` definition:

These functions allow the onboard LEDs to flash red to indicate an error.

Next, in the `setup()` function:

The serial communication is initialized at a baud rate of 115200.

putting the board into power-saving mode) is disabled to avoid communication problems with the magnetometer.

Error and event handlers are initialized.

NDP processor is initialized; this process includes populating the external Flash memory of the board with the NDP processor's internal microcontroller firmware (`mcu_fw_120_v91.synpkg`) and the NDP processor's internal DSP firmware (`dsp_firmware_v91.synpkg`).

The BMM150 sensor is initialized; this includes setting it into normal operation with an output data rate (ODR) of 10 Hz.

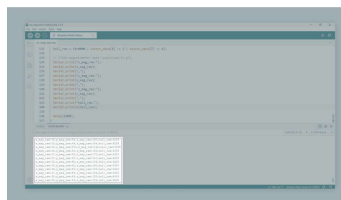
Finally, in the `loop()` function:

Memory is allocated for the sensor data; data is then read from the sensor and stored in this allocated space.

Raw sensor data is then extracted and parsed into raw magnetometer data. Raw sensor data is extracted from the `sensor_data` array, 8-bits at a time, and then combined to form a 16-bit integer for each axis (X, Y, Z) and raw Hall resistance value.

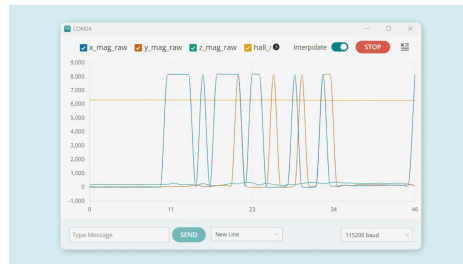
Raw magnetometer data is printed on the Serial Monitor, allowing the user to observe sensor data in real-time.

After uploading the example code, you should see the magnetometer data on the IDE's Serial Monitor, as shown below:



Nicla Voice onboard raw magnetometer data on the IDE's Serial Monitor

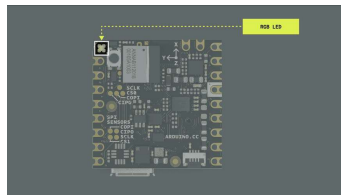
After a while, you should see a real-time graph showing raw data from the board's onboard magnetometer, as shown below (move the board):



Actuators

RGB LED

The Nicla Voice features a built-in I2C RGB LED that can be a visual feedback indicator for the user. The LED is connected through the boards' I2C port; therefore, specific functions must be used to operate the LED colors.



Nicla Voice built-in RGB LED

To use the RGB LED, include the

`Nicla_System` header:

```
1 // Include the Nicla System header
2 #include "Nicla_System.h"
```

Since the functions are scoped under a class name called `Nicla`, you must explicitly write it before each statement. To initialize the board's built-in RGB LED along with the Nicla system inside the

```
1 void setup() {  
2   // Initialize the Nicla sys  
3   nicla::begin();  
4   nicla::leds.begin();  
5 }
```

The built-in RGB LED can be set to a desired color using the RGB color model in which the red, green, and blue primary colors of light are added together in various ways to reproduce a broad array of colors. There are predefined colors:

`red`, `green`, `blue`, `yellow`, `magenta`, and `cyan`; to turn off the LED, use `off`. To set the built-in RGB LED to a predefined color, for example, green or blue:

```
1 // Set the LED color to green  
2 nicla::leds.setColor(green);  
3 delay(1000);  
4  
5 // Set the LED color to blue  
6 nicla::leds.setColor(blue);  
7 delay(1000);
```

To turn off the built-in RGB LED:

```
1 // Turn off the LED  
2 nicla::leds.setColor(off);
```

You can also choose a value between 0 and 255 for each color component (red, green, or blue) to set a custom color:


```
1 // Define custom color values
2 int red = 234;
3 int green = 72;
4 int blue = 122;
5
6 // Set the LED to the custom color
7 nicla::leds.setColor(red, green, blue);
8 delay(1000);
9
10 // Turn off the LED and wait
11 nicla::leds.setColor(off);
12 delay(1000);
```

Here you can find a complete example code to blink the built-in I2C RGB LED of the Nicla Voice:

```
1 // Include the Nicla System
2 #include "Nicla_System.h"
3
4 void setup() {
5     // Initialize the Nicla system
6     nicla::begin();
7     nicla::leds.begin();
8 }
9
10 void loop() {
11     // Set the LED color to red
12     nicla::leds.setColor(red);
13     delay(1000);
14
15     // Turn off the LED and wait
16     nicla::leds.setColor(off);
17     delay(1000);
18 }
```

Communication

This section of the user manual covers the different communication protocols that are supported by the Nicla Voice board, including the Serial Peripheral Interface (SPI), Inter-Integrated Circuit (I2C), Universal Asynchronous Receiver-Transmitter (UART), and Bluetooth® Low

section. The Nicla Voice features dedicated pins for each communication protocol, making connecting and communicating with different components, peripherals, and sensors easy.

SPI

The Nicla Voice supports SPI communication, which allows data transmission between the board and other SPI-compatible devices. The pins used in the Nicla Voice for the SPI communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
CS / P0_29	SS or 6
COP1 / P0_27	SCK or 8
CIP0 / P0_28	MOSI or 7
SCLK / P0_11	MISO or 9

Please, refer to the [board pinout section](#) of the user manual to find them on the board.

Include the `SPI` library at the top of your sketch to use the SPI communication protocol. The SPI library provides functions for SPI communication:

```
1 #include <SPI.h>
```

In the `setup()` function, initialize the SPI library, define and configure the chip select (`CS`) pin:

```
1 void setup() {
2   // Set the chip select pin
3   pinMode(SS, OUTPUT);
4
5   // Pull the CS pin HIGH to
6   digitalWrite(SS, HIGH);
7
8   // Initialize the SPI comm
9   SPI.begin();
10 }
```

To transmit data to an SPI-compatible device, you can use the following commands:

```
1 // Replace with the target c
2 byte address = 0x00;
3
4 // Replace with the value to
5 byte value = 0xFF;
6
7 // Pull the CS pin LOW to se
8 digitalWrite(SS, LOW);
9
10 // Send the address
11 SPI.transfer(address);
12
13 // Send the value
14 SPI.transfer(value);
15
16 // Pull the CS pin HIGH to u
17 digitalWrite(SS, HIGH);
```

I2C

The Nicla Voice supports I2C communication, which allows data transmission between the board and other I2C-compatible devices. The pins used in the Nicla Voice for the I2C communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
---------------------	---------------------

Microcontroller Pin	Arduino Pin Mapping
P0_22	I2C_SDA or 4

Please, refer to the [board pinout section](#) of the user manual to find them on the board. The I2C pins are also available through the onboard ESLOV connector of the Nicla Voice.

To use I2C communication, include the `Wire` library at the top of your sketch. The `Wire` library provides functions for I2C communication:

```
1 #include <Wire.h>
```

In the `setup()` function, initialize the I2C library:

```
1 // Initialize the I2C communication
2 Wire.begin();
```

To transmit data to an I2C-compatible device, you can use the following commands:

```
1 // Replace with the target c
2 byte deviceAddress = 0x1;
3
4 // Replace with the appropri
5 byte instruction = 0x00;
6
7 // Replace with the value to
8 byte value = 0xFF;
9
10 // Begin transmission to the
11 Wire.beginTransmission(device
12
13 // Send the instruction byte
14 Wire.write(instruction);
15
16 // Send the value
17 Wire.write(value);
18
19 // End transmission
20 Wire.endTransmission();
```

To read data from an I2C-compatible device, you can use the `requestFrom()` function to request data from the device and the `read()` function to read the received bytes:

```
1 // The target device's I2C a
2 byte deviceAddress = 0x1;
3
4 // The number of bytes to re
5 int numBytes = 2;
6
7 // Request data from the tar
8 Wire.requestFrom(deviceAddre
9
10 // Read while there is data
11 while (Wire.available()) {
12   byte data = Wire.read();
13 }
```

UART

The pins used in the Nicla Voice for the UART communication protocol are the following:

Microcontroller Pin	Arduino Pin Mapping
P0_09	SERIAL1_TX or 1
P0_20	SERIAL1_RX or 2

Please, refer to the [board pinout section](#) of the user manual to find them on the board.

To begin with UART communication, you'll need to configure it first. In the `setup()` function, set the baud rate (bits per second) for UART communication:

```
1 // Start UART communication
2 Serial1.begin(9600);
```

To read incoming data, you can use a `while()` loop to continuously check for available data and read individual characters. The code shown above stores the incoming characters in a String variable and process the data when a line-ending character is received:

```
1 // Variable for storing incoming data
2 String incoming = "";
3
4 void loop() {
5     // Check for available data on serial
6     while (Serial1.available()) {
7         // Allow data buffering to complete
8         delay(2);
9         char c = Serial1.read();
10
11         // Check if the character is a newline
12         if (c == '\n') {
13             // Process the received data
14             processData(incoming);
15
16             // Clear the incoming string
17             incoming = "";
18         } else {
19             // Add the character to the string
20             incoming += c;
21         }
22     }
23 }
```

To transmit data to another device via UART, you can use the `write()` function:

```
1 // Transmit the string "Hello world!"
2 Serial1.write("Hello world!");
```

You can also use the `print` and `println()` to send a string without a newline character or followed by a newline character:

```
1 // Transmit the string "Hello world!"
2 Serial1.print("Hello world!");
3
4 // Transmit the string "Hello world!" followed by a newline
5 Serial1.println("Hello world!");
```

To enable Bluetooth® Low Energy communication on the Nicla Voice, you can use the [ArduinoBLE library](#).

Here is an example of how to use the ArduinoBLE library to create a voltage level monitor application:

```
1  #include "Nicla_System.h"
2  #include <ArduinoBLE.h>
3
4
5  // Define the voltage serv
6  BLEService voltageService(
7  BLEUnsignedCharCharacteris
8
9
10 const int analogPin = A0;
11
12
13 /**
14  Read voltage level from a
15  then maps the voltage rea
16
17
18  @param none
19  @return the voltage level
20  */
21 int readVoltageLevel() {
22   int voltage = analogRead(
23   int voltageLevel = map(vo
24   return voltageLevel;
25 }
26
27
28 void setup() {
```

The example code shown above creates a Bluetooth® Low Energy service and characteristic for transmitting a voltage value read by the analog pin A0 of the Nicla Voice to a central device Bluetooth® device like a smartphone or another microcontroller.

The code begins by importing all the necessary libraries and defining the Bluetooth® Low Energy service and characteristics.

In the `setup()` function, the code initializes the Nicla Voice board and

begins advertising the defined Bluetooth® Low Energy service.

A Bluetooth® Low Energy connection is constantly verified in the `loop()` function, being the built-in LED in red while looking for a connection. When a central device connects to the Nicla Voice, its built-in LED will change its color to blue. The code then enters into a loop that constantly reads the voltage level from an analog input and maps it to a percentage value between 0 and 100. The voltage level is printed to the Serial Monitor and transmitted to the central device over the defined Bluetooth® Low Energy characteristic.

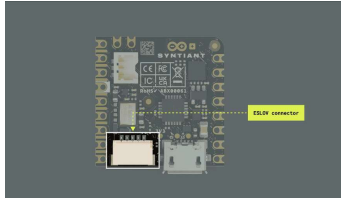
You can use the [nRF Connect for Mobile](#) app from Nordic Semiconductor to test the functionality of the example code shown below. nRF Connect is a powerful tool that allows you to scan and explore Bluetooth® Low Energy devices and communicate with them.



Bluetooth® Low Energy service and characteristic information from a Nicla Voice device

ESLOV Connector

The Nicla Voice board features an onboard ESLOV connector meant as an **extension** of the I2C communication bus. This connector simplifies connecting various sensors, actuators, and other modules to the Nicla Voice without soldering or wiring.



Nicla Voice built-in ESLOV connector

The ESLOV connector is a small 5-pin connector with a 1.00 mm pitch; the mechanical details of the connector can be found in the connector's datasheet.

The pin layout of the ESLOV connector is the following:

- 1 VCC_IN (5V input)
- 2 INT
- 3 SCL
- 4 SDA
- 5 GND

The manufacturer part number of the ESLOV connector is SM05B-SRSS and its matching receptacle manufacturer part number is SHR-05V-S-B.

Support

If you encounter any issues or have questions while working with the Nicla Voice, we provide various support resources to help you find answers and solutions.

Help Center

Explore our [Help Center](#), which offers a comprehensive collection of articles and guides for the Nicla Voice. The Arduino Help Center is designed to provide in-depth technical assistance and help you make the most of your device.

[Nicla Family help center page](#)

Forum

Join our community forum to connect with other Nicla Voice users, share your experiences, and ask questions. The forum is an excellent place to learn from others, discuss issues, and discover new ideas and projects related to the Nicla Voice.

[Nicla Voice category in the Arduino Forum](#)

Contact Us

Please get in touch with our support team if you need personalized assistance or have questions not covered by the help and support resources described before. We're happy to help you with any issues or inquiries about the Nicla Voice.

[Contact us page](#)

Suggest changes

The content on [docs.arduino.cc](#) is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino](#)
[Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

